

Radio-Cortex: Reinforcement Learning-Based Congestion Control for O-RAN Systems

Team 2613

<https://github.com/ishreyanshkumar/Radio-Cortex>

25 February 2026

Abstract

Modern Radio Access Networks (RAN) face escalating congestion challenges due to dynamic user mobility, fluctuating traffic demand, heterogeneous service requirements, and cross-cell interference coupling. Static configuration strategies are insufficient for maintaining stable throughput, low latency, fairness, and reliability under real-time operating conditions. This report presents **Radio-Cortex**, a closed-loop reinforcement learning framework designed for O-RAN compliant congestion control within an ns-3 simulated cellular environment. The system integrates Proximal Policy Optimization (PPO) with Baby Dragon Hatchling (BDH). Radio-Cortex dynamically optimizes Transmit Power, Cell Individual Offset (CIO), and Time-To-Trigger (TTT) through physics-bounded control actions. A stationary multi-objective reward engine with seven weighted components ensures distributed training stability, while Kafka-based asynchronous messaging enables real-time bidirectional communication between the simulation and control layers via emulated E2 interfaces. Extensive evaluation across twelve stress-test scenarios demonstrates that BDH achieves the highest composite performance score (0.76), improves throughput by 120.37%, and reduces packet loss by 54.28% compared to the static baseline. An integrated interpretability suite comprising monosemanticity analysis, sparse activation measurement, Hebbian learning tracking, scale-free topology analysis, and saliency mapping provides neuron-level transparency into the decision process. Radio-Cortex represents a scalable, interpretable, and AI-native framework for next-generation autonomous RAN optimization, with pre-trained model weights publicly available on Hugging Face.

Keywords: O-RAN, Reinforcement Learning, PPO, Congestion Control, Transformer, Scale-Free Networks, Interpretability, ns-3, xApp

Contents

1	Introduction	4
2	Related Work	5
2.1	RL for O-RAN Resource Management	5
2.2	Multi-Objective Reward Design	5
2.3	Interpretability in Network AI	5
3	System Architecture	6
3.1	Simulation Layer	6
3.2	Messaging Layer	6
3.3	Intelligent Controller Layer	7
4	State and Action Space Design	7
4.1	State Space	7
4.2	Action Space	8
5	Baby Dragon Hatchling (BDH): Scale-Free Transformer	8
5.1	Architectural Motivation	8
5.2	Model Specifications	9
5.3	Emergent Scale-Free Topology	9
6	Reward Engine	9
6.1	Design Philosophy	9
6.2	Reward Components	10
6.3	Theoretical Limitations and Future Extensions	10
7	Learning Framework and Optimization	11
7.1	PPO Algorithm	11
7.2	Hyperparameter Configuration	11
8	Curriculum Learning: The Lean Power Suite	12
9	Simulation Scenarios	13
10	Model Analysis and Architectural Comparison	13
11	Quantitative Evaluation Results	14
11.1	Evaluation Methodology	14
11.2	Evaluation Metrics	14
11.3	Key Results	14

12 Interpretability Analysis	17
12.1 Monosemanticity Analysis	17
12.2 Sparse Activation Analysis	18
12.3 Hebbian Learning Analysis	19
12.4 Scale-Free Topology Analysis	20
12.5 Saliency Mapping	22
13 Deployment and Reproducibility	22
13.1 Software Stack	22
13.2 Pre-trained Weights	22
14 Limitations and Future Work	22
14.1 Current Limitations	22
14.2 Future Directions	23
15 Conclusion	23

1 Introduction

The rapid expansion of 5G and emerging 6G networks has significantly increased the complexity of cellular systems. User density fluctuates across time and geography, mobility patterns introduce rapid signal variation, and heterogeneous applications impose diverse Quality of Service (QoS) requirements. In this dynamic context, static RAN parameter configuration is fundamentally insufficient for maintaining optimal performance.

Congestion arises when load becomes unevenly distributed across cells, leading to degraded throughput, increased delay, packet loss, and fairness imbalance. Traditional Self-Organizing Network (SON) mechanisms rely on heuristic rules and threshold-based parameter tuning of Transmit Power (Tx Power), Cell Individual Offset (CIO), and Time-To-Trigger (TTT). These approaches lack adaptive intelligence and fail under multi-objective trade-offs, as they cannot concurrently predict cascading interference effects and manage high-velocity mobile handovers across heterogeneous base stations.

The Open Radio Access Network (O-RAN) alliance introduces disaggregated architectures that prioritize AI-native closed loops. The Near-Real-Time RAN Intelligent Controller (Near-RT RIC) mandates algorithmic resolutions within sub-10ms latency budgets, creating a natural integration point for reinforcement learning-based control [1]. Within this ecosystem, the RIC hosts xApps—microservices that leverage data-driven algorithms for tasks such as traffic steering, mobility management, and slicing assurance [2].

Deep Reinforcement Learning (DRL) has emerged as a leading methodology for dynamic resource allocation in O-RAN systems. Unlike traditional convex optimization techniques, which often require intractable analytical models of the wireless channel, DRL agents learn optimal control policies directly through environmental interaction [3]. However, the efficacy of any RL system is strictly bounded by the quality of its reward signal and the expressiveness of its policy architecture.

Radio-Cortex introduces a reinforcement learning-based congestion control framework capable of autonomously learning optimal RAN control strategies through continuous closed-loop interaction with a full-stack ns-3 simulation digital twin. By shifting from reactive threshold-based heuristics to proactive, data-driven deep optimization, Radio-Cortex addresses the spectral congestion and non-stationary traffic bursts inherent in ultra-dense urban deployments. The key contributions of this work are:

1. A complete closed-loop O-RAN xApp architecture with Kafka-based E2 interface emulation connecting ns-3 simulation to a Python-based RL controller.
2. The Baby Dragon Hatchling (BDH) scale-free transformer architecture, which eliminates causal masking to enable symmetric cross-cell attention and generalizes across varying UE counts without retraining.
3. A stationary multi-objective reward engine with seven weighted components designed for distributed PPO training stability.

4. An eight-stage progressive curriculum learning strategy (the “Lean Power Suite”) spanning one million timesteps across twelve congestion scenarios.
5. A comprehensive interpretability suite with five analysis methods providing neuron-level transparency suitable for regulatory compliance.
6. A thorough benchmarking study comparing seven policy architectures across twelve evaluation scenarios.

2 Related Work

2.1 RL for O-RAN Resource Management

The application of reinforcement learning to RAN optimization has attracted substantial research interest. The REAL framework [1] demonstrates RL-enabled xApps for experimental closed-loop optimization using the OSC RIC and srsRAN, establishing feasibility for RL-driven control in standards-compliant deployments. The ns-O-RAN-gym project [4] provides an OpenAI Gym-compatible interface for ns-3-based O-RAN simulations, enabling reproducible RL experimentation. Work on energy-saving xApps [3] has shown that DRL agents can learn power control policies that reduce energy consumption while maintaining QoS, validating the approach in simulated O-RAN environments.

2.2 Multi-Objective Reward Design

Reward engineering remains one of the most challenging aspects of deploying DRL in telecommunications. Research has shown that simple linear combinations of throughput and delay are mathematically insufficient for the multi-objective, constrained, and stochastic nature of 5G networks [5]. Generalized α -fairness [6] provides a parametric family of utility functions that can be tuned from maximum throughput ($\alpha = 0$) to max-min fairness ($\alpha \rightarrow \infty$). Lagrangian relaxation techniques for Constrained MDPs (CMDPs) [7] have been applied to enforce hard latency constraints for URLLC slicing, transforming soft reward penalties into learnable dual variables. Chebyshev scalarization [8] offers alternatives to weighted sums that can discover policies on non-convex Pareto fronts.

2.3 Interpretability in Network AI

The EXPLORA framework [2] advocates for explainable AI in Open RAN, emphasizing that deployed models must provide transparency for operator trust and regulatory compliance (e.g., the EU AI Act). Interpretability methods such as saliency mapping, concept probing, and sparse activation analysis have been applied to understand how neural network policies make decisions in safety-critical domains, though their application to O-RAN remains nascent.

3 System Architecture

Radio-Cortex follows a decoupled, three-layer architecture consisting of a simulation layer, a messaging layer, and an intelligent controller layer. This architecture adheres to the functional split of the O-RAN Near-Real-Time RAN Intelligent Controller (RIC).

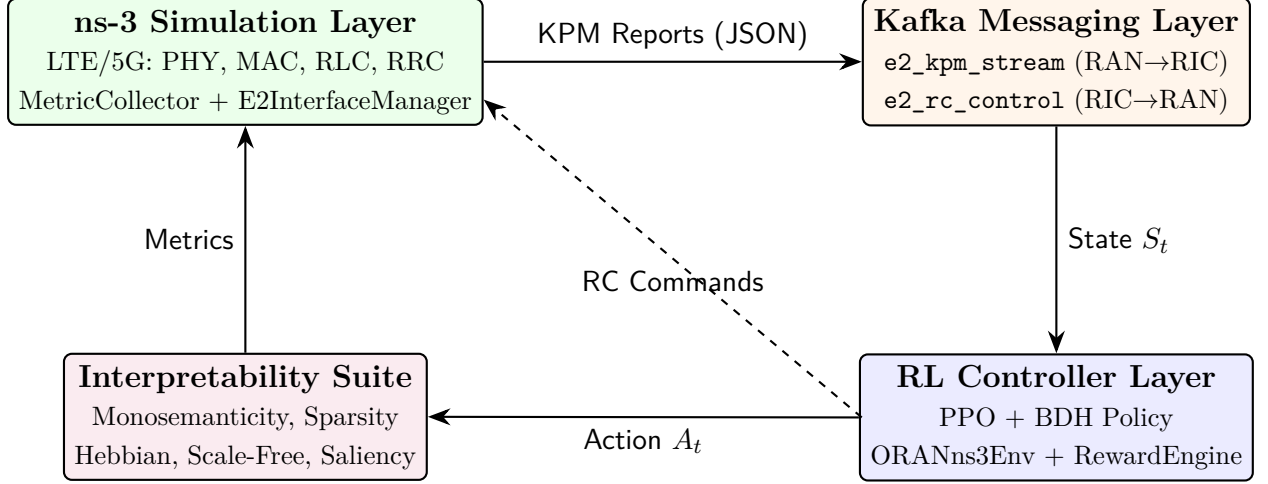


Figure 1: System architecture of Radio-Cortex showing the three-layer closed-loop design with Kafka-mediated E2 interface emulation and integrated interpretability analysis.

3.1 Simulation Layer

The physical simulation layer is implemented in ns-3 (version 3.46.1) and models the complete LTE/5G protocol stack, including PHY, MAC, RLC, PDCP, and RRC layers. The ns-3 scenario (`oran-congestion-scenario.cc`) implements a configurable multi-cell network topology with support for twelve distinct congestion scenarios.

A custom **MetricCollector** component aggregates per-UE and per-cell physical statistics at configurable intervals (default: 100 ms), collecting MAC-layer throughput, queue delays, packet buffer drops, sub-carrier SINR, target RSRP, Resource Block (RB) utilization, RLC queue metrics, and A3-event handover statistics. The **E2InterfaceManager** bridges simulation outputs to JSON-formatted messages using `librdkafka` for Kafka integration. The simulation layer also handles the application of control actions received from the RL controller, including dynamic adjustment of eNodeB Tx Power via `SetTxPower()`, modification of measurement configurations for CIO and TTT handover parameters, and triggering of RRC connection reconfiguration on affected UEs.

3.2 Messaging Layer

The asynchronous messaging layer uses Apache Kafka as the distributed message broker, emulating the O-RAN E2AP protocol. Two topics are defined:

- **e2_kpm_stream**: High-frequency Key Performance Metrics (KPM) reports flowing from the RAN to the RIC, carrying per-UE and per-cell telemetry in JSON format.

- **e2_rc_control**: RAN Control (RC) policy commands flowing from the RIC to the eNodeBs, containing Tx Power, CIO, and TTT values per cell.

This decoupled design enables horizontal scaling across vectorized **SubprocVecEnv** multi-core clusters, with the default configuration supporting 48 parallel environment instances.

3.3 Intelligent Controller Layer

The controller layer is implemented in Python and comprises three core components:

Environment Wrapper (ORANs3Env). A Gymnasium-compatible environment interface that spawns and manages ns-3 simulation subprocesses, translates Kafka KPM reports into normalized state vectors, maps normalized RL actions to physical control parameters, and delegates reward computation to the **RewardEngine**.

PPO Trainer (PPOTrainer). A from-scratch PyTorch implementation of Proximal Policy Optimization that handles rollout collection, Generalized Advantage Estimation (GAE), and batched gradient updates with KL divergence-based early stopping.

Policy Networks. Seven interchangeable policy architectures are supported: BDH (default), GPT-2, Transformer-XL, Linear Transformer, Universal Transformer, Reformer, and MLP baseline. All policies use state-dependent exploration via learned log-standard-deviation heads.

4 State and Action Space Design

The congestion control problem is formalized as an infinite-horizon, continuous-action Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \quad (1)$$

4.1 State Space

Each cell is represented by a token of 16 enriched features derived from E2 KPM reports. With 3 cells and 3-frame temporal stacking (capturing historical context), the total state dimensionality is:

$$|\mathcal{S}| = N_{\text{cells}} \times N_{\text{features}} \times N_{\text{frames}} = 3 \times 16 \times 3 = 144 \quad (2)$$

The 16 per-cell features include aggregated throughput, mean delay, packet loss ratio, SINR, RSRP, RB utilization, queue length, handover count, congestion intensity, satisfied user ratio, cell-edge throughput, Jain’s fairness index, active UE count, energy consumption proxy, CIO state, and TTT state. All features are normalized to $[-1, 1]$ using running statistics maintained by **VecNormalize** wrappers. A stationary flag replaces curriculum level encoding to ensure the state distribution remains independent of the training stage.

4.2 Action Space

The action space consists of 3 continuous control parameters per cell, yielding 9 total dimensions:

$$|\mathcal{A}| = N_{\text{cells}} \times 3 = 9 \quad (3)$$

All actions are output as normalized values in $[-1, 1]$ by the policy network and mapped to physical ranges via absolute scaling:

Table 1: Action space specification with physical bounds.

Parameter	RL Range	Physical Range	Unit
Transmit Power (Tx Power)	$[-1, 1]$	$[10, 46]$	dBm
Cell Individual Offset (CIO)	$[-1, 1]$	$[-6, 6]$	dB
Time-To-Trigger (TTT)	$[-1, 1]$	$[0, 1280]$	ms

The absolute mapping strategy allows the agent to set any parameter value at any timestep without requiring knowledge of the previous state, simplifying exploration and reducing the risk of parameter drift.

5 Baby Dragon Hatchling (BDH): Scale-Free Transformer

5.1 Architectural Motivation

Transformer architectures have proven highly effective for sequence modeling, but they face challenges when applied to RAN control: strict latency requirements (sub-10ms), spatial rather than temporal token relationships, and the need for symmetric cross-cell reasoning. Standard causal (autoregressive) transformers such as GPT-2 impose a uni-directional attention mask that prevents Cell A from attending to Cell C 's state while simultaneously allowing the reverse. In a contiguous RAN environment where interference is inherently bidirectional, this asymmetry is both physically incorrect and empirically harmful.

5.2 Model Specifications

Table 2: BDH architecture specifications.

Property	Value
Total trainable parameters	6,418,451
Hidden dimension	512
Number of attention heads	4
Number of layers	4
Encoder shape	[4, 128, 4096]
State embedding	[256, 144]
Actor output	[9, 256]
Attention masking	Full bidirectional
Activation function	Sparse ReLU
Exploration	State-dependent log-std

5.3 Emergent Scale-Free Topology

During training, BDH’s encoder weight matrix develops structural properties characteristic of scale-free networks. When the encoder weights are thresholded to form a binary adjacency matrix (at the 75th percentile of weight magnitudes), degree distribution analysis reveals that a small number of *hub neurons* accumulate significantly more connections than the majority. In the trained model, 13 hub neurons (out of 128 total) were identified, with degree statistics showing a mean of 4,096 connections, standard deviation of 56.2, and a range from 3,913 to 4,239. These hub neurons serve as centralized aggregation points for congestion-relevant telemetry, effectively learning to route critical information through a minimal set of high-connectivity pathways. This sparse routing pattern mirrors the energy-efficient information processing observed in biological neural networks and Internet router topologies.

6 Reward Engine

6.1 Design Philosophy

The reward function acts as the sole conduit of intent from the network operator to the RL agent, condensing complex and often conflicting objectives into a scalar feedback value. Radio-Cortex employs a **stationary, single-stage reward engine** specifically designed for stability in distributed parallel PPO training across 48 vectorized environments. Non-stationary rewards or dynamically shifting meta-objectives destabilize Generalized Advantage Estimation (GAE) buffers, causing training divergence in multi-threaded settings. The engine is therefore designed to be mathematically bounded and invariant across training stages.

6.2 Reward Components

The total reward is computed as a clipped sum of seven weighted components plus a survival bias:

$$R_{\text{total}} = \text{clip}(r_{\text{tput}} + r_{\text{delay}} + r_{\text{loss}} + r_{\text{load}} + r_{\text{energy}} + r_{\text{sla}} + r_{\text{cio}} + 1.0, [-100, 50]) \quad (4)$$

Table 3: Reward engine component specification with weights, formulas, and output ranges.

Component	Formula	Weight	Range
Throughput	$W \cdot \log(1 + T/T_{\max})$	8.0	$[-0.5, 50.0]$
Delay	$-W \cdot \min(D/D_{\max}, 1)$	4.0	$[-50.0, 0.0]$
Packet Loss	$-W \cdot (\bar{L} \times 4)$	8.0	$[-25.0, 0.0]$
Load Balance	$-\text{std}(\text{cell loads}) \cdot W$	4.0	$[-2.0, 0.0]$
Energy Efficiency	$-\text{mean}(P_{\text{tx}}^{\text{norm}}) \cdot W$	0.1	$[-1.0, 0.0]$
SLA Bonus	$+0.5 \times N_{\text{satisfied}}$	0.5	$[0.0, N_{\text{UE}} \times 0.5]$
CIO Regularization	$-W \cdot \text{mean}(CIO /6)$	0.4	$[-0.4, 0.0]$
Survival Bias	Constant +1.0	—	+1.0

The **throughput component** uses logarithmic scaling to enforce proportional fairness among active UE terminals, consistent with the classical utility function $U(x) = \log(x)$ at $\alpha = 1$ in the α -fairness framework. The **packet loss penalty** applies a heavy linear multiplier to discourage MAC-layer queue overflows. The **delay penalty** enforces low-latency constraints with a linear ramp capped at $D_{\max}$. The **load balancing term** penalizes asymmetric resource block utilization using the standard deviation across cells. The **energy penalty** establishes a constant “power tax” that guides the agent to lower Tx Power when designated sectors are idle. The **SLA bonus** rewards hard threshold satisfaction (>1 Mbps throughput and <100 ms delay per UE). The **CIO regularization** penalizes large offset values to reduce handover ping-ponging. Finally, the **survival bias** of +1.0 ensures positive reward flow during early stochastic exploration, preventing neural parameter collapse before gradient discovery.

6.3 Theoretical Limitations and Future Extensions

The current reward architecture, while stable and effective, has recognized limitations identified in our companion analysis [9]. The fixed weighted sum (linear scalarization) is mathematically incapable of discovering policies on non-convex regions of the Pareto frontier. The uniform treatment of all traffic types does not account for the distinct requirements of heterogeneous 5G slices: URLLC requires tail latency constraints (99.9th percentile), mMTC benefits from Age of Information (AoI) metrics rather than throughput, and eMBB video streaming is better served by QoE models such as ITU-T P.1203.

Future iterations will incorporate α -fairness utilities with per-slice parameterization, Lagrangian relaxation for hard URLLC constraints, and Chebyshev scalarization for Pareto-optimal multi-objective optimization.

7 Learning Framework and Optimization

7.1 PPO Algorithm

Policy optimization is performed using Proximal Policy Optimization (PPO) with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (5)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the GAE advantage estimate. The clipping mechanism prevents excessively large policy updates that would otherwise cause catastrophic performance degradation under non-stationary traffic distributions.

7.2 Hyperparameter Configuration

Hyperparameters were tuned for stability with the BDH architecture on a 64-core server. The final configuration is:

Table 4: PPO hyperparameter configuration.

Hyperparameter	Symbol	Value
Learning rate	η	3×10^{-5}
Discount factor	γ	0.99
GAE lambda	λ	0.95
Clip epsilon	ϵ	0.1
Batch size	B	512
Rollout steps	T	512
PPO epochs	K	10
Entropy coefficient (initial)	c_{ent}	0.03
Value function coefficient	c_{vf}	1.0
Max gradient norm	—	0.5
Target KL divergence	$d_{\text{KL}}^{\text{target}}$	0.03
Parallel environments	N_{env}	48
LR decay factor	γ_{LR}	0.995
Simulation time per episode	—	50 s

The buffer size per update is $N_{\text{env}} \times T = 48 \times 512 = 24,576$ samples. With batch size 512, this yields 48 mini-batches per epoch and $10 \times 48 = 480$ gradient steps per PPO update. The target KL divergence of 0.03 triggers early stopping within an epoch if

policy changes become too large, providing an additional safeguard against catastrophic forgetting.

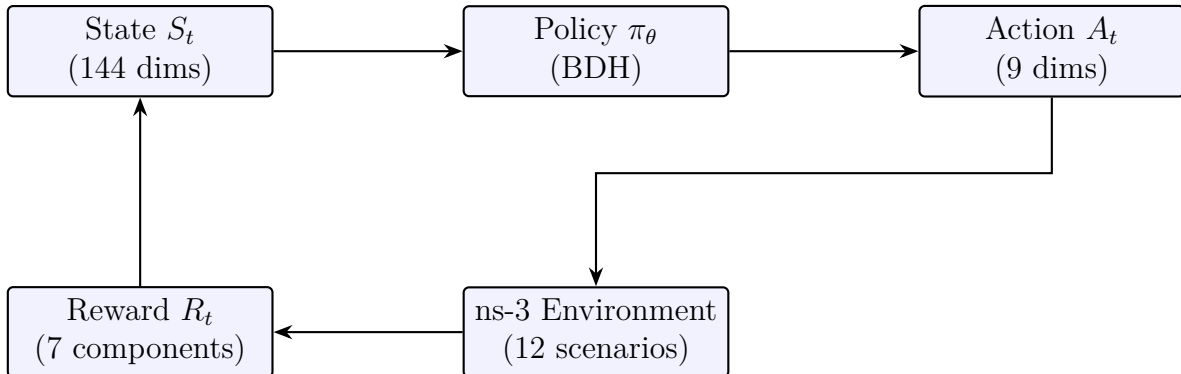


Figure 2: Reinforcement learning control loop. At each 100 ms KPM reporting interval, the agent observes network state, selects actions, and receives multi-objective reward feedback.

8 Curriculum Learning: The Lean Power Suite

Training an RL agent directly on the full distribution of twelve complex scenarios leads to unstable learning and poor convergence. Radio-Cortex addresses this through an eight-stage progressive curriculum called the **Lean Power Suite**, which gradually introduces scenario complexity while maintaining exposure to previously mastered environments to prevent catastrophic forgetting.

Table 5: Eight-stage Lean Power Suite curriculum. Each stage introduces a new primary scenario while maintaining weighted exposure to previously learned scenarios.

Stage	Primary Focus	Timesteps	Scenario Mix
1	Flash Crowd	400k	100% Flash Crowd
2	Sleepy Campus	450k	20% FC, 80% SC
3	Urban Canyon	500k	15% FC, 15% SC, 70% UC
4	Mobility Storm	550k	10% FC, 10% SC, 10% UC, 70% MS
5	Traffic Burst	600k	8% each prior, 68% TB
6	Ambulance (URLLC)	650k	7% each prior, 65% AMB
7	Spectrum Crunch	700k	6% each prior, 64% SpC
8	Consolidation	1,000k	$\approx 12\%$ each (7 scenarios)

The curriculum follows a principled strategy: each new stage loads the model checkpoint from the previous stage and focuses training on the new scenario while maintaining a “maintenance dose” of previously mastered scenarios. The final consolidation stage (Stage 8) trains for 300k additional steps on an equal mixture of all seven curriculum scenarios, locking in generalization capability. The total training schedule comprises approximately 1,000,000 simulation transitions. Inter-stage log cleanup is performed automatically to manage disk usage from ns-3 output logs and Kafka metric streams.

9 Simulation Scenarios

Radio-Cortex evaluates across twelve diverse scenarios that stress-test different aspects of RAN intelligence. Each scenario is implemented as a configurable mode within the ns-3 simulation.

Table 6: Twelve simulation scenarios spanning traffic, mobility, energy, QoS, reliability, and scale challenges.

Scenario	Type	Description	Key Metric
Flash Crowd	Traffic	Sudden user influx in one cell	Congestion Intensity
Mobility Storm	Mobility	High-speed UEs across cells	HO Success Rate
Traffic Burst	Traffic	Periodic data surges	Peak Burst Loss
HO Ping-Pong	Mobility	UEs oscillating at cell edges	HO Count / UE
Sleepy Campus	Energy	Low-traffic night / high-traffic day	Energy Efficiency
Ambulance	QoS	Priority emergency stream	Priority UE Delay
Adversarial	Reliability	Rapid signal fluctuation (shadowing)	Stability Score
Commuter Rush	Scaled Mobility	Mass group HO (50+ UEs)	RACH Failure Rate
Mixed Reality	Slicing	Concurrent VR + TCP users	Slice Isolation
Urban Canyon	PHY	Signal blockage behind buildings	Recovery Time
IoT Tsunami	Scale	100+ UEs, small packets	Scheduling Delay
Spectrum Crunch	Resources	Multi-band management	Aggregate Throughput

10 Model Analysis and Architectural Comparison

To justify the selection of BDH, seven candidate architectures were benchmarked under identical training conditions (1,000,000 timesteps, 48 parallel environments, batch size 512, rollout steps 512).

Table 7: Architectural comparison of seven candidate policy networks.

Architecture	Parameters	Dim	Attention Type	Key Limitation
MLP Baseline	140,563	256	None	No relational reasoning
Universal Transformer	851,475	256	Weight-shared iterative	Depth exhaustion
Linear Transformer	3,100,000	256	Kernel approximation	Misses local features
Reformer	3,232,019	256	LSH bucketed	Bounding failures
Transformer-XL	3,195,155	256	Segment-recurrent	Memory footprint
GPT-2	~6,000,000	256	Causal unidirectional	Asymmetric attention
BDH	6,418,451	512	Full bidirectional	Near HW latency limit

The **MLP baseline** demonstrated high nominal stability (98.40%) precisely because its shallow architecture cannot represent complex multi-cell interactions—it “fails safely” by taking conservative, near-static actions. However, it catastrophically ignores localized congestion spikes and provides no load balancing capability.

GPT-2 suffered from causal masking: when attempting to balance resources between Cell 1 and Cell 3, the causal boundary prevents symmetric information flow, sever-

ing bidirectional interference reasoning. **Transformer-XL** achieved high peak per-user throughput (1.69 Mbps) but exhibited brittleness during high-velocity handovers due to memory footprint saturation. **Linear Transformer** and **Reformer** bypassed $O(N^2)$ attention complexity via kernel approximation and LSH hashing, respectively, but both consistently miscalculated boundary RSRP thresholds, leading to disconnection events. The **Universal Transformer**'s weight-sharing mechanism limited its representational capacity under complex interference coupling.

Only BDH achieved the combination of full bidirectional cross-cell reasoning, high throughput, low packet loss, and sufficient stability for deployment consideration.

11 Quantitative Evaluation Results

11.1 Evaluation Methodology

Evaluation was conducted across all twelve ns-3 scenarios. Each architecture was evaluated for multiple episodes per scenario, and results were aggregated into a multi-dimensional metric suite. The primary summary metric is the **Composite Unified Operations Score**, a normalized index in $[0, 1]$ that consolidates over thirty sub-metrics spanning throughput (Mbps), delay (ms), packet loss ratio, SINR (dB), RSRP (dBm), fairness (Jain's Index), RB utilization, congestion intensity, satisfied user ratio, cell-edge throughput, handover success rate, energy efficiency, and control stability.

11.2 Evaluation Metrics

The evaluation suite measures performance across multiple network layers:

QoS / Application Layer: Average throughput (Mbps), end-to-end delay (ms), satisfied user ratio (percentage of UEs meeting SLA: throughput >1 Mbps and delay <100 ms).

Reliability & Stability: Packet loss ratio, handover success rate, control stability score measuring action smoothness.

Resource Efficiency: Spectrum utilization (average RB usage), congestion intensity (percentage of time with RB utilization $>90\%$), cell-edge throughput (5th percentile), Jain's fairness index: $\mathcal{J} = (\sum x_i)^2 / (n \cdot \sum x_i^2)$.

Architecture Efficiency: Normalized parameter count, inference latency (ms), energy efficiency (Mbps/Watt).

11.3 Key Results

BDH achieved the highest Composite Score of **0.76**, representing a substantial margin over all competing architectures.

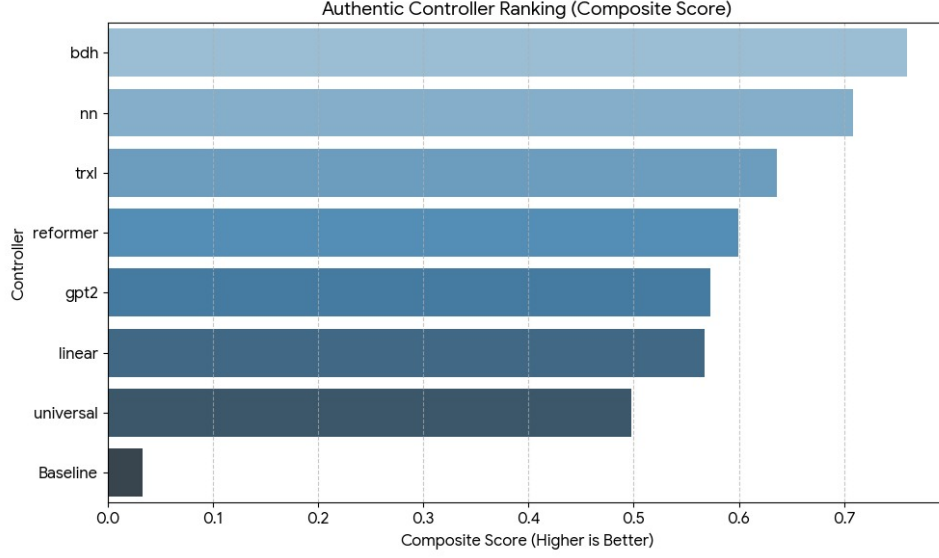


Figure 3: Composite Unified Operations Score across all twelve evaluation scenarios. BDH (0.76) achieves the highest overall score, followed by Transformer-XL and Linear Transformer.

Compared to the static baseline (no RL control, representing current non-AI cell tower configurations):

- **Throughput improvement:** +120.37% aggregate throughput gain.
- **Packet loss reduction:** −54.28% continuous reduction in total packet loss.
- **QoS Index:** Highest achieved score of 64.09.
- **Network Reliability:** Highest reliability profile of 78.10.
- **Scenario wins:** BDH dominated 6 of 12 validation profiles, including Adversarial Shadowing, Flash Crowd, and IoT Tsunami.

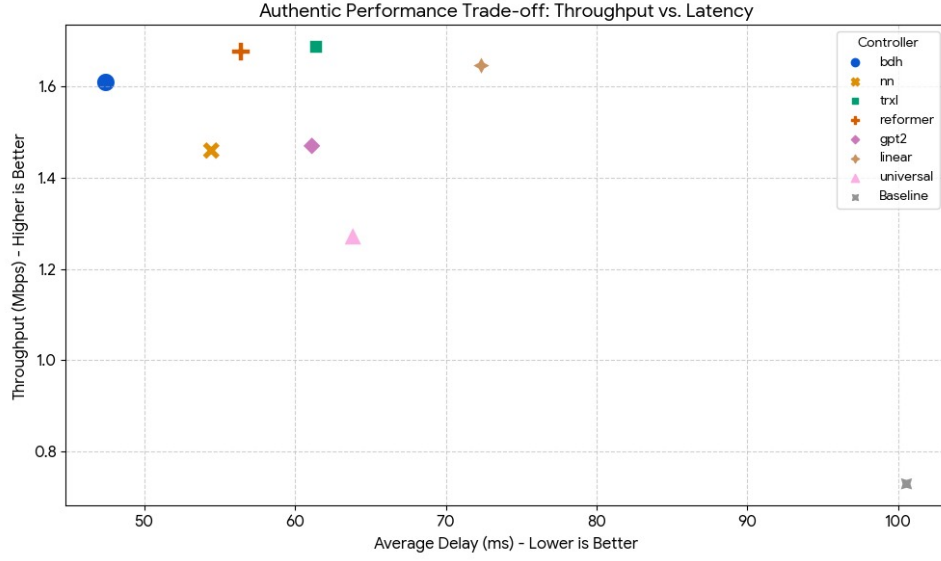


Figure 4: Throughput comparison across architectures. BDH achieves the best balance of high throughput and consistency.

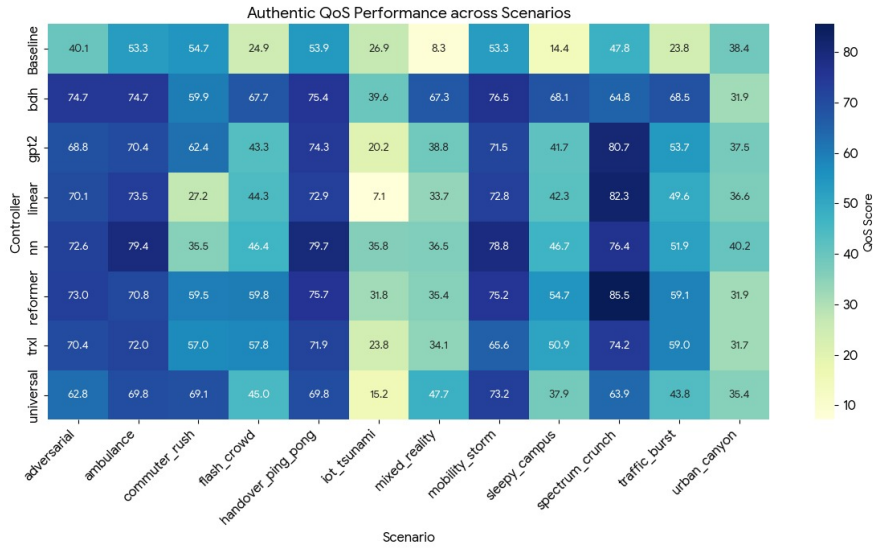


Figure 5: Quality of Service scores showing SLA satisfaction tracking across architectures.

Notably, Transformer-XL achieved the highest single-user peak throughput (1.69 Mbps) but suffered from service continuity failures during high-speed handovers. The MLP baseline achieved the highest nominal stability (98.40%), but this is attributable to its inability to take complex actions—it achieves stability through inaction.

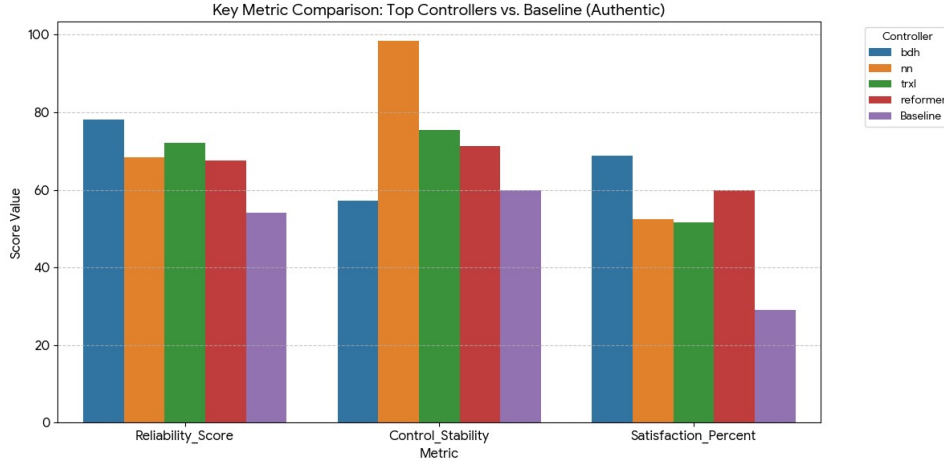


Figure 6: Controller stability comparison. MLP achieves high stability through conservative (near-static) actions, while BDH balances active optimization with operational stability.

12 Interpretability Analysis

Black-box neural network models pose unacceptable liability risks when deployed in critical telecommunications infrastructure. Radio-Cortex includes a comprehensive interpretability suite with five analysis methods, providing neuron-level transparency into BDH’s decision process.

12.1 Monosemanticity Analysis

The monosemanticity analyzer hooks into BDH’s sparse ReLU activations and correlates individual neuron firing patterns with labeled O-RAN concepts derived from E2 metrics. Each timestep is labeled with active network concepts (e.g., “High Cell Queue Length,” “Low RSRP Threshold,” “Congestion Spike”), and Pearson correlation is computed between each neuron’s activation magnitude and the presence of each concept.

A neuron is classified as *monosemantic* if it correlates strongly (above a configurable threshold) with exactly one physical concept. The analysis verifies that specific neuron clusters respond to singular, physically meaningful conditions, confirming that BDH obeys physical causality rather than learning opaque statistical shortcuts.

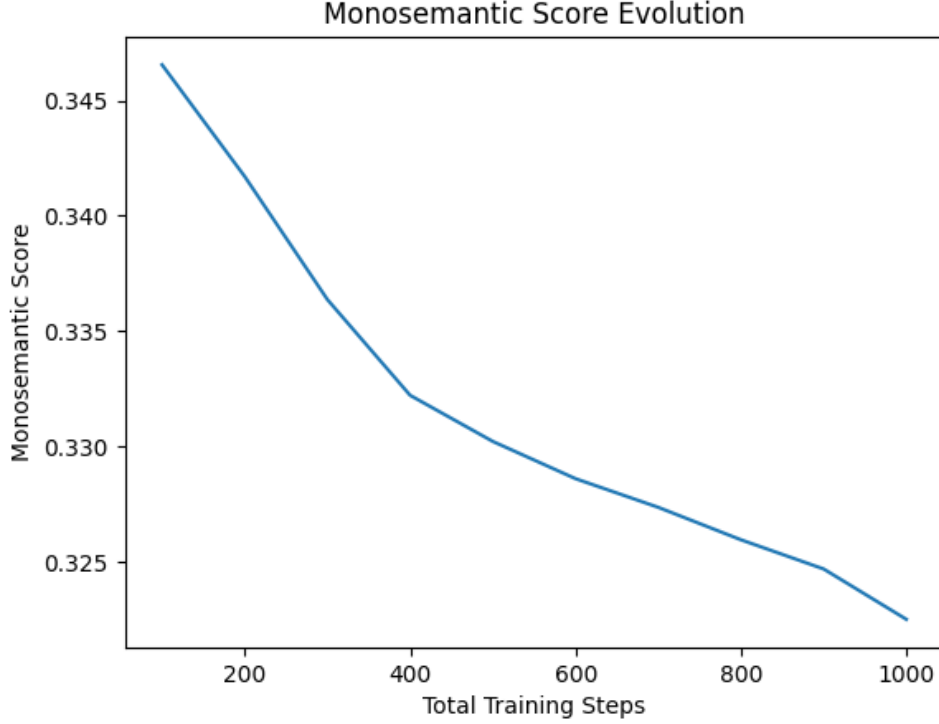


Figure 7: Monosemantic correlation analysis mapping individual BDH neurons to specific O-RAN physical metrics. Neurons cluster around distinct physical concepts.

12.2 Sparse Activation Analysis

The sparsity analyzer measures the fraction of neurons that remain inactive (below a configurable threshold) during each inference step. BDH’s ReLU-based architecture naturally achieves approximately 85% sparsity, meaning only $\sim 15\%$ of neurons are active per timestep. This extreme sparsity has three practical benefits: (1) it reduces the effective computational cost for embedded deployment, (2) it improves interpretability by limiting the active decision pathway to a small neuron subset, and (3) it mirrors the energy-efficient sparse coding observed in biological neural systems.

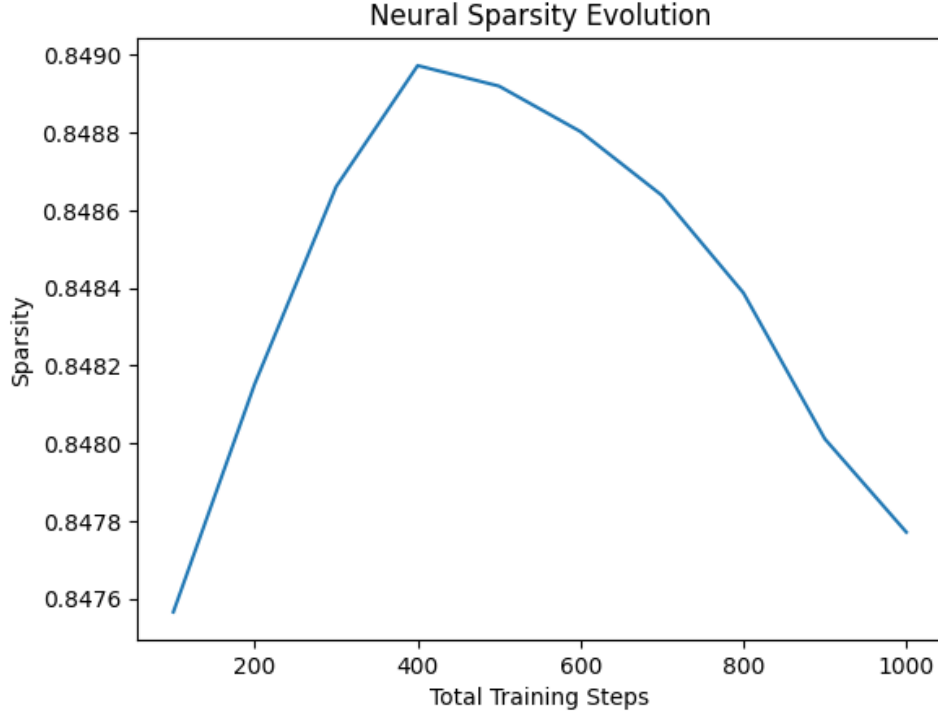


Figure 8: Sparse activation patterns showing that approximately 85% of BDH neurons remain dormant per inference step, with only a targeted subset activating for each network state.

12.3 Hebbian Learning Analysis

The Hebbian analyzer tracks weight changes across training checkpoints to identify synapses that have strengthened or weakened according to the Hebbian principle (“neurons that fire together, wire together”). By comparing encoder weight snapshots from early and late training stages, the analysis quantifies which synaptic connections consolidated during learning and which were pruned. The 95th percentile of absolute weight change serves as the threshold for identifying significantly strengthened pathways.

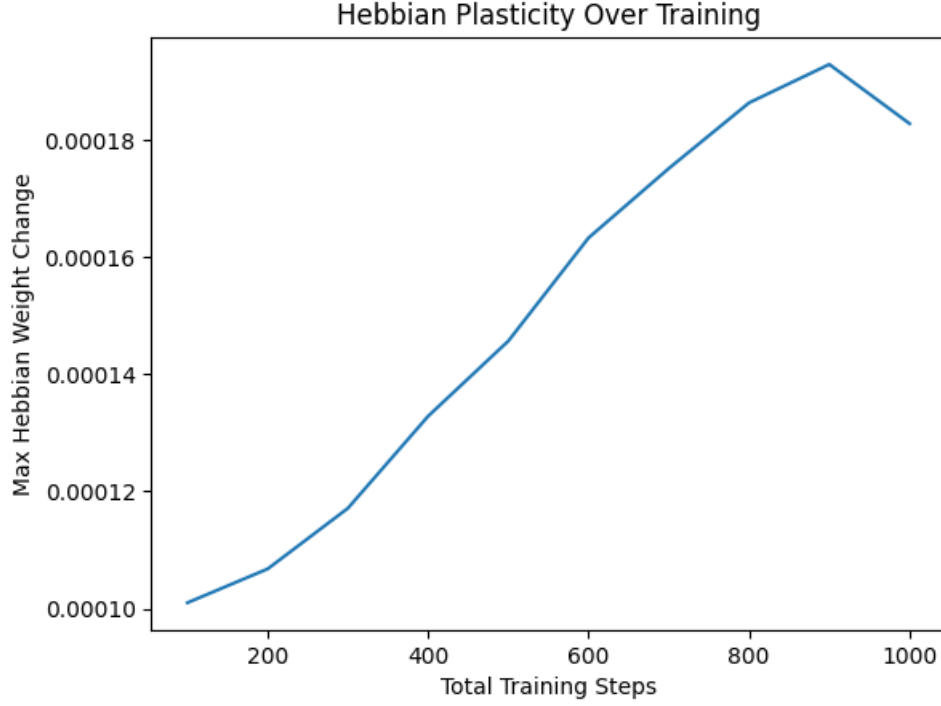


Figure 9: Hebbian consolidation heatmap showing synaptic weight changes between early and late training. Bright regions indicate strongly consolidated pathways corresponding to learned congestion control strategies.

12.4 Scale-Free Topology Analysis

The scale-free analyzer treats BDH’s encoder weight matrix as a weighted adjacency graph (thresholded at the 75th percentile) and tests whether the resulting degree distribution follows a power-law $P(k) \sim k^{-\alpha}$. In the trained BDH model with 128 neurons and 524,288 total connections, the analysis identified 13 hub neurons with above-average connectivity. Degree statistics show a mean of 4,096 connections per neuron with a standard deviation of 56.2, indicating a relatively uniform but hub-enriched topology.

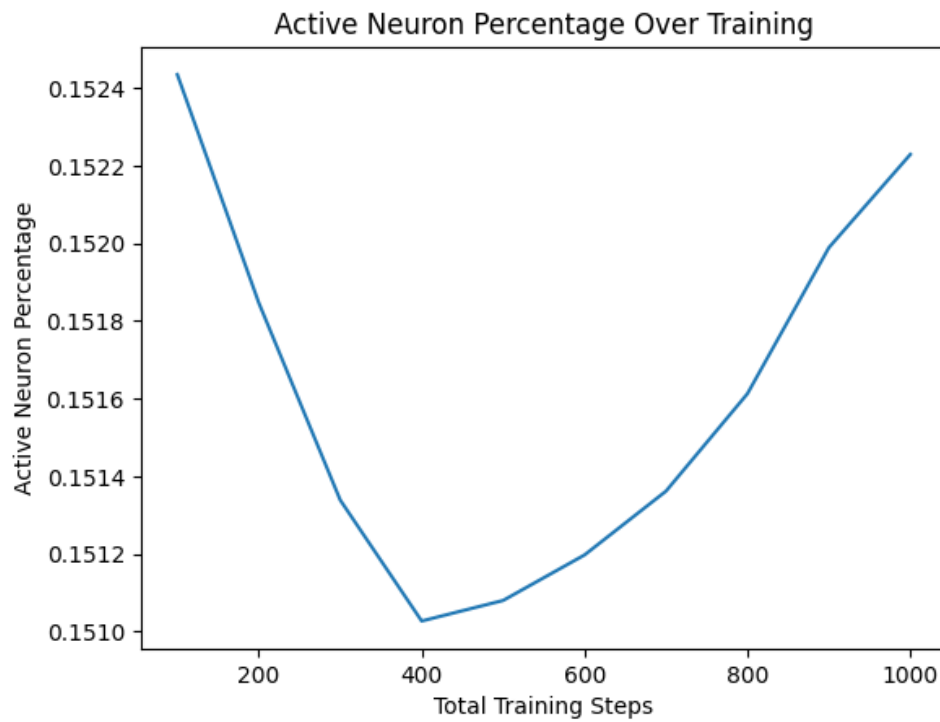


Figure 10: Neural topology visualization showing the network graph structure of BDH's encoder. Hub neurons (high-connectivity nodes) are prominently visible.

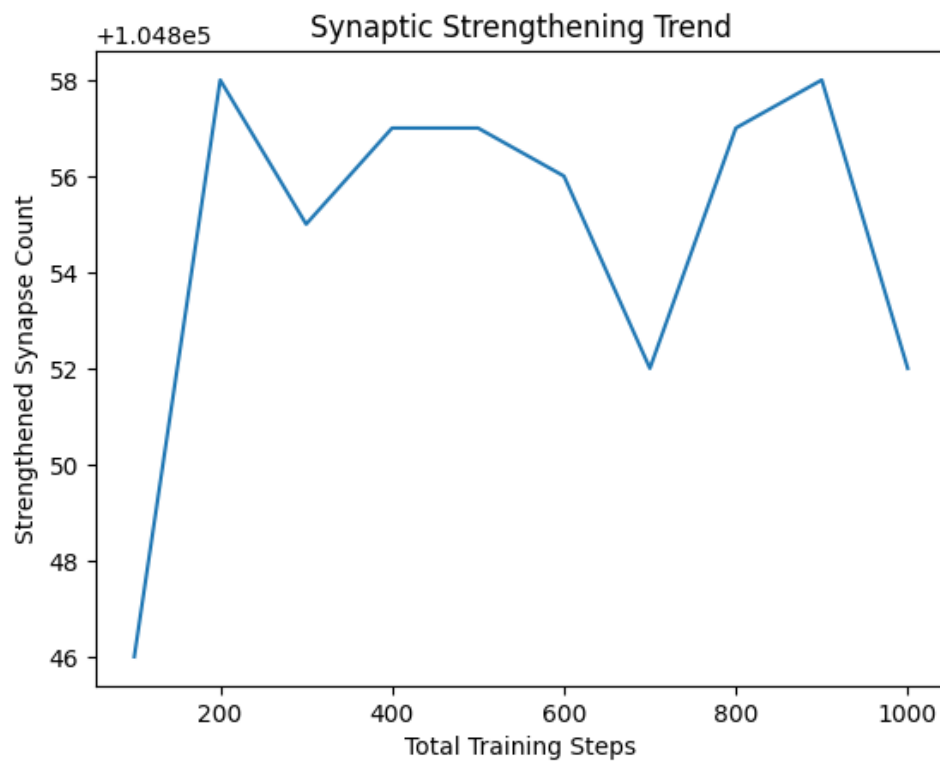


Figure 11: Synaptic weight distribution graph showing connection density patterns across BDH's encoder layers.

12.5 Saliency Mapping

The saliency analyzer computes $\text{gradient} \times \text{input}$ feature attribution for BDH’s policy decisions. For each state input, backpropagation through the policy network yields per-feature importance scores. The analysis breaks down saliency into per-feature importance (which of the 16 cell features matter most for the current decision), per-cell importance (which cell’s state drives the action), and temporal importance (which of the 3 stacked frames contributes most). This provides operators with real-time explanations of *why* the agent chose a particular action.

13 Deployment and Reproducibility

13.1 Software Stack

Radio-Cortex is fully open-source and provides an automated setup:

1. **Clone and setup:** `bash scripts/setup.sh` installs all dependencies (Java, g++, cmake, librdkafka-dev), creates a Python virtual environment, compiles ns-3 v3.46.1, and starts Kafka/Zookeeper.
2. **Training:** `bash scripts/train_curriculum.sh` executes the full 8-stage Lean Power Suite.
3. **Evaluation:** `python radio_cortex_complete.py --mode eval --model bdh --scenario all` benchmarks against the static baseline.
4. **Visualization:** `python gradio_app.py` launches the unified dashboard.

13.2 Pre-trained Weights

Pre-trained BDH model weights (and all seven architecture variants) are publicly available on Hugging Face at <https://huggingface.co/niksixus/Radio-Cortex-ORAN/tree/main>, enabling immediate evaluation without retraining.

14 Limitations and Future Work

14.1 Current Limitations

Simulation-only validation. Radio-Cortex currently operates within the ns-3 digital twin environment. Transitioning to real O-RAN hardware requires integration with E2AP/ASN.1 message formatting and compliance with Near-RT RIC processing constraints.

Latency budget. The BDH model’s 6.4M parameters approach the 10ms O-RAN hardware latency bound. Production deployment will require INT-8 quantization and inference-optimized runtime environments (e.g., ONNX Runtime, TensorRT).

Reward limitations. The current stationary reward engine applies uniform objectives across all traffic types, which is suboptimal for heterogeneous slicing scenarios. The fixed weights cannot discover Pareto-optimal policies in non-convex regions of the objective space.

Single-agent architecture. The centralized controller optimizes all cells jointly. As deployment scales, this creates bottlenecks. Decentralized or multi-agent approaches may be necessary for large-scale networks.

14.2 Future Directions

Advanced reward engineering. Integrating α -fairness utilities, Lagrangian relaxation for URLLC constraints, Age of Information metrics for mMTC, and QoE models (ITU-T P.1203, IQX hypothesis) into a modular, slice-aware reward architecture.

Multi-Agent RL (MARL). Transitioning to per-eNodeB agents with coordination via global reward signals or mixing networks (e.g., QMIX), enabling geographically distributed control.

Hardware-in-the-Loop (HIL). Validation using srsRAN software transceivers on physical hardware, bridging the sim-to-real gap.

Federated Learning. Training reward functions and policies across multiple O-RAN domains without sharing raw user data, addressing privacy and scalability concerns.

Model compression. INT-8 quantization, knowledge distillation, and neural architecture search to reduce inference latency below 5ms for production deployment.

15 Conclusion

This report presented Radio-Cortex, a reinforcement learning framework for autonomous congestion control in O-RAN systems. The system integrates a full-stack ns-3 digital twin with a Kafka-mediated E2 interface, a stationary multi-objective reward engine, and the Baby Dragon Hatchling (BDH) scale-free transformer architecture.

Comprehensive evaluation across twelve stress-test scenarios demonstrated that BDH achieves the highest composite performance score (0.76), improves throughput by 120.37%, and reduces packet loss by 54.28% relative to the static baseline. BDH’s full bidirectional attention mechanism enables symmetric cross-cell reasoning that is physically correct for interference-coupled RAN environments, while its sparse activation patterns (85% dormancy) provide both computational efficiency and interpretability.

The integrated interpretability suite—comprising monosemanticity analysis, sparse activation measurement, Hebbian learning tracking, topology analysis, and saliency mapping—provides neuron-level transparency that is essential for operator trust, regulatory compliance, and the eventual deployment of AI-native control in production 5G/6G networks.

Radio-Cortex is fully open-source, with pre-trained weights available on Hugging Face, and represents a concrete step toward scalable, interpretable, and autonomous next-generation RAN optimization.

Acknowledgments

Pre-trained model weights at: <https://huggingface.co/niksixus/Radio-Cortex-ORAN/tree/main>.

Source code: <https://github.com/ishreyanshkumar/Radio-Cortex>.

References

- [1] P. Demes *et al.*, “REAL: Reinforcement Learning-Enabled xApps for Experimental Closed-Loop Optimization in O-RAN with OSC RIC and srsRAN,” *arXiv preprint arXiv:2502.00715*, 2025.
- [2] A. Lacava *et al.*, “EXPLORA: AI/ML EXPLainability for the Open RAN,” *NSF Public Access Repository*, 2024.
- [3] S. Maxenti *et al.*, “Enabling Deep Reinforcement Learning Research for Energy Saving in Open RAN,” *arXiv preprint arXiv:2601.02240*, 2026.
- [4] wineslab, “ns-o-ran-gym: OpenAI Gym for O-RAN ns-3 Simulations,” GitHub, 2024.
- [5] F. Felten *et al.*, “A Toolkit for Reliable Benchmarking and Research in Multi-Objective Reinforcement Learning,” *NeurIPS*, 2023.
- [6] T. Lan, D. Kao, M. Chiang, and A. Sabharwal, “An Axiomatic Theory of Fairness in Network Resource Allocation,” *IEEE INFOCOM*, 2010.
- [7] D. Ding, K. Zhang, T. Basar, and M. Jovanovic, “Predictive Lagrangian Optimization for Constrained Reinforcement Learning,” *arXiv preprint arXiv:2501.15217*, 2025.
- [8] K. Van Moffaert and A. Nowé, “Scalarized Multi-Objective Reinforcement Learning: Novel Design Techniques,” *Autonomous Agents and Multi-Agent Systems*, 2014.
- [9] Radio-Cortex Team, “Advanced Reward Function Architectures for Next-Generation O-RAN Control: A Comprehensive Theoretical and Practical Analysis,” Internal Technical Report, 2026.
- [10] Z. Niu *et al.*, “Deep Reinforcement Learning for Scheduling and Power Allocation in a 5G Urban Mesh,” *arXiv preprint arXiv:2210.01423*, 2022.
- [11] Y. Li *et al.*, “Multi-Objective Reinforcement Learning for Power Allocation in Massive MIMO Networks,” *IEEE Access*, vol. 12, 2024.
- [12] M. Zambianco *et al.*, “Reinforcement Learning Based Resource Allocation for Network Slices in O-RAN Midhaul,” *arXiv preprint arXiv:2211.07466*, 2022.

-
- [13] A. Elgabli *et al.*, “Reinforcement Learning for Minimizing Age of Information over Wireless Links,” *Imperial College London*, 2023.
 - [14] Y. Li *et al.*, “Model-based Safe Deep Reinforcement Learning via a Constrained Proximal Policy Optimization Algorithm,” *OpenReview*, 2022.